

Architecture de Déploiement

Dev → Staging → Production VPS Oracle Cloud

Render
API

Neon
PostgreSQL

Flutter
Mobile

GitHub
CI/CD

Vercel
Web

Oracle
VPS Cible

Environnements Dev · Staging · Production (Oracle VPS cible)

Composants : API Laravel · PostgreSQL · Flutter App · Web Blade · CI/CD

Chapitres : 8 — Vue globale · Services · Déploiement · CI/CD · MAJ · Backup · Migration · Sécurité

Horizon : Phase dev = Render+Neon · Phase prod = Oracle VPS self-hosted

TABLE DES MATIÈRES

01	Vue d'ensemble globale	Diagramme complet · 3 environnements · Flux de données
02	Les services actuels (Phase Dev)	Render · Neon · GitHub CI · Vercel · Flutter Stores
03	Déploiement de l'API Laravel	Process complet · Variables d'environnement · Health checks
04	Base de données PostgreSQL	Neon tech · Connexions · Pool · Migrations · Schéma tenant
05	Application Flutter Mobile	Build · Signing · Distribution Stores · OTA Updates
06	Pipeline CI/CD GitHub Actions	Workflow complet · Stages · Tests · Deploy automatique
07	Mises à jour — Architecture complète	API · App mobile · Web · Sans coupure de service
08	Backup & Restauration	Stratégie 3-2-1 · PostgreSQL dump · Fichiers · Test restore
09	Migration vers VPS Oracle Cloud	Pourquoi Oracle · Plan de migration · Checklist · Timeline
10	Sécurité & Secrets	Variables d'env · Chiffrement · Accès · Rotation des secrets

CH.01

VUE D'ENSEMBLE GLOBALE

3 environnements · Tous les composants · Flux de données

01

Ce document décrit l'architecture de déploiement de Leopardo RH en deux phases : la phase de développement actuelle (Render + Neon + GitHub CI + Vercel) et la cible de production (VPS Oracle Cloud self-hosted). Les deux architectures sont documentées complètement pour faciliter la migration.

Les 3 environnements

Environnement	URL	Infrastructure	Déclencheur de déploiement	Qui y accède ?
Development (local)	localhost:8000 localhost:3000	Docker Compose (api + postgres + redis local)	Manuel (php artisan serve ou docker compose up)	Développeur uniquement
Staging	api-staging.leopardo-rh.com app-staging.leopardo-rh.com	Render (web service) Neon (postgres branch staging)	Push sur branche staging (GitHub Actions auto)	Équipe dev + bêta testeurs
Production	api.leopardo-rh.com app.leopardo-rh.com	Render → migre vers Oracle VPS (cible)	Merge sur main (GitHub Actions + approbation manuelle)	Clients réels

Architecture globale actuelle (Phase Dev)



Architecture actuelle Phase Dev — Render + Neon + GitHub CI. Cible = VPS Oracle (voir Chapitre 09).

Flux de données entre composants

Flux	Source	Destination	Protocole	Auth
Requêtes API mobile	App Flutter	Render Web Service (Laravel)	HTTPS/REST	Sanctum token Bearer
Requêtes dashboard web	Blade + Alpine.js	Render Web Service (Laravel)	HTTPS/REST + Sessions	Sanctum session cookie
DB queries	Laravel API	Neon PostgreSQL	PostgreSQL Wire Protocol (TLS)	SSL cert + password
Upload fichiers (reçus, PDFs)	Laravel API	Storage local (Render disk) ou S3 compatible	HTTPS PUT	API Key S3
Envoi emails	Laravel Queue	Mailtrap (staging) / SMTP prod	SMTP TLS	User + Password SMTP
Notifications push	Laravel Queue	Firebase FCM (Android) APNs (iOS)	HTTPS	Service Account JSON / APNs cert
Téléchargement app	Utilisateur final	Play Store / App Store	HTTPS	N/A (public)
Deploy API	GitHub Actions	Render (deploy hook)	HTTPS webhook	Render Deploy Key
Backup DB	Render Cron Job (ou Oracle Cron)	S3 / Object Storage	HTTPS	S3 Key

CH.02

LES SERVICES ACTUELS — PHASE DÉVELOPPEMENT

Render · Neon · GitHub · Vercel · Flutter Stores

02

Inventaire des services

RENDER

Render

Hébergement API Laravel

DEV+PROD

- Auto-deploy depuis GitHub
- Starter 7\$/mois (prod)
- Zero-downtime deployments

NEON

Neon.tech

PostgreSQL serverless

DEV+PROD

- PostgreSQL 16 serverless
- Branching staging/prod
- PgBouncer inclus

GH

GitHub

Code + CI/CD

DEV+PROD

- Repository + GitHub Actions
- Tests, build, deploy auto
- Secrets management

VERCEL

Vercel

Dashboard web (optionnel)

DEV

- Si dashboard séparé de l'API
- Pour l'instant sur Render
- CDN edge global

PLAY

Google Play Store

Distribution Android

DEV+PROD

- 25\$ compte une fois
- Build AAB signé keystore
- Internal → Beta → Prod

APPLE

Apple App Store

Distribution iOS

DEV+PROD

- 99\$/an Apple Developer
- Build macOS XCode requis
- TestFlight pour la beta

FCM

Firebase

Notifications push

DEV+PROD

- FCM Android + APNs iOS
- Gratuit jusqu'à 1M msgs
- Analytics inclus

MAIL

Mailtrap

Emails (dev/staging)

DEV

- Intercepte tous les emails
- Sandbox gratuit
- SMTP prod via Postmark

Coûts estimés Phase Dev vs Production

Service	Plan actuel (dev)	Coût/mois	Plan production	Coût/mois prod
Render (API)	Free (sleep après 15min)	0 €	Starter	7 \$/mois
Neon (PostgreSQL)	Free (0.5GB, 1 branch)	0 €	Launch (10GB, branching)	19 \$/mois
GitHub	Free (public + private)	0 €	Free ou Team	0-4 \$/mois
Vercel	Hobby (si utilisé)	0 €	Pro (si utilisé)	20 \$/mois
Firebase (FCM)	Spark (gratuit)	0 €	Spark jusqu'à 1M notifs	0 €

Service	Plan actuel (dev)	Coût/mois	Plan production	Coût/mois prod
Google Play Store	25 \$ (une fois)	—	Inclus	—
Apple App Store	99 \$/an	8 €/mois	99 \$/an	8 €/mois
Mailtrap (staging)	Free (1000 emails/mois)	0 €	SMTP prod réel (Postmark)	10 \$/mois
TOTAL estimé	Phase développement	~8 €/mois	Phase staging+prod (avant Oracle)	~68 \$/mois
TOTAL cible	Migration Oracle VPS	—	Oracle Always Free tier	~0-20 \$/mois

Oracle Cloud Free Tier (Always Free) : 2 vCPU + 1GB RAM ARM + 50GB stockage + 10TB de bande passante/mois. Pour Leopardo RH en phase initiale, c'est suffisant pour faire tourner Laravel + Nginx + PostgreSQL sur un seul VPS. Aucun coût jusqu'à un volume significatif.

CH.03

DÉPLOIEMENT DE L'API LARAVEL

Render actuel · Process complet · Variables · Health checks

03

Configuration Render actuelle

```
render.yaml
# render.yaml – configuration déclarative du service Render
services:
  - type: web
    name: leopardo-rh-api
    runtime: php
    region: frankfurt          # Europe – marché cible MENA + France
    plan: starter              # $7/mois – pas de sleep mode
    buildCommand: composer install --no-dev --optimize-autoloader &&
                        php artisan config:cache &&
                        php artisan route:cache &&
                        php artisan view:cache
    startCommand: php artisan serve --host=0.0.0.0 --port=$PORT
    healthCheckPath: /api/health
    envVars:
      - key: APP_ENV
        value: production
      - key: APP_KEY
        generateValue: true      # Render génère automatiquement
      - key: DATABASE_URL
        fromDatabase:           # Lié à la DB Neon
          name: leopardo-rh-db
          property: connectionString
      - key: MEDIAMTX_INTERNAL_SECRET
        sync: false             # À renseigner manuellement dans Render Dashboard
```

Variables d'environnement — inventaire complet

Variable	Valeur type	Sensible	Source	Utilisée par
APP_NAME	Leopardo RH	Non	Code (hardcoded)	Emails, logs
APP_ENV	production / staging	Non	Render env	Laravel config
APP_KEY	base64:xxx (32 chars)	Oui — Chiffrement	Render auto-generate	Crypt, sessions, cookies
APP_URL	https://api.leopardo-rh.com	Non	Render env	URLs générées
DATABASE_URL	postgresql://user:pass@host/db	Oui — DB access	Neon dashboard	Eloquent, migrations
CACHE_DRIVER	file (dev) / redis (prod)	Non	Code	Cache Laravel

Variable	Valeur type	Sensible	Source	Utilisée par
QUEUE_CONNECTION	sync (dev) / database (prod)	Non	Code	Jobs async
MAIL_MAILER	smtp	Non	Code	Mailer Laravel
MAIL_HOST / PORT / USER / PASS	smtp.mailtrap.io / 2525...	Oui	Mailtrap/SMTP prod	Mail::send()
FIREBASE_CREDENTIALS_JSON	{ service account JSON }	Oui — Push notifs	Firebase console	NotificationService
MEDIAMTX_BASE_URL	https://proxy.leopardo-rh.com	Non	Manuel	StreamTokenService
MEDIAMTX_INTERNAL_SECRET	secret_token_hex	Oui	Généré manuellement	Auth MediaMTX
CAMERA_STREAM_TOKEN_TTL	14400 (4h en secondes)	Non	Code	StreamTokenService
OPENAI_API_KEY / ANTHROPIC_API_KEY	sk-xxx...	Oui — IA externe	Provider dashboard	AiChatService
AI_PROVIDER	openai ou anthropic	Non	Code	AiChatService
S3_KEY / S3_SECRET / S3_BUCKET	xxx	Oui — Stockage	Provider S3	Storage::put()
TURN_URL / TURN_USER / TURN_PASS	turn:xxx:3478 / xxx	Oui	coturn config	WebRTC caméras

Health check endpoint

```
GET /api/health — endpoint Render health check
// routes/api.php — endpoint public sans auth
Route::get('/health', function () {
    return response()->json([
        'status' => 'ok',
        'timestamp' => now()->toIso8601String(),
        'version' => config('app.version'), // depuis .env APP_VERSION
        'env' => config('app.env'),
        'db' => DB::connection()->getPdo() ? 'connected' : 'error',
        'cache' => Cache::has('health_check') || Cache::put('health_check', 1, 60),
    ]);
});
// Render ping ce endpoint toutes les 30s
// Si HTTP != 200 pendant 3 pings → restart automatique du service
```

Process de déploiement API étape par étape

1. **Push sur GitHub** → déclenche le workflow GitHub Actions (voir Ch.06).
2. **Tests Pest** → tous les tests doivent passer. Si échec → déploiement bloqué.

3. **Build sur Render** → composer install --no-dev + cache config/route/view.
4. **php artisan migrate --force** → migrations lancées automatiquement sur la DB Neon.
5. **Render lance le nouveau container** → démarre le service sur le nouveau code.
6. **Health check** → Render attend que /api/health retourne 200 avant de couper l'ancien container.
7. **Zero-downtime** → Render garde l'ancien container en vie pendant le démarrage du nouveau. Les requêtes en cours finissent sur l'ancien. Les nouvelles arrivent sur le nouveau.

Durée moyenne d'un déploiement sur Render : 2 à 4 minutes. Pendant ce temps, les requêtes sont servies par l'ancienne version sans interruption. Les migrations doivent être backward-compatible (pas de DROP COLUMN ou RENAME pendant qu'une version tourne).

CH.04

BASE DE DONNÉES POSTGRESQL

Neon.tech · Connexions · Pool · Migrations · Multi-tenancy

04

Neon.tech — configuration actuelle

Paramètre	Valeur (dev/staging)	Valeur (prod cible)
Provider	Neon.tech (serverless)	Neon.tech Pro ou Oracle VPS PostgreSQL
Version PostgreSQL	16	16
Région	EU West (Frankfurt)	EU West + Oracle Oracle Frankfurt
Connection pooling	PgBouncer (inclus Neon)	PgBouncer (inclus Neon) ou pgBouncer VPS
Pool mode	Transaction mode	Session mode (pour les prepared statements)
Max connections pool	10 (free) / 100 (launch)	100 (launch) / illimité (VPS)
Max connections DB	107 (free)	10 000 (VPS PostgreSQL)
Branches	main + staging	main + staging + feature branches (dev)
SSL	Obligatoire	Obligatoire
Backup automatique	Oui (Neon gère)	Oui (scripts + Neon ou pg_dump VPS)
Point-in-time recovery	7 jours (launch)	30 jours (VPS + WAL archiving)

Configuration Laravel — connexion DB

```
config/database.php + .env
# .env — connexion Neon via DATABASE_URL
DATABASE_URL=postgresql://leopardo:password@ep-xxx.eu-central-1.aws.neon.tech/leopardo_rh?sslmode=require

# config/database.php — parsing de l'URL
'pgsql' => [
    'driver' => 'pgsql',
    'url' => env('DATABASE_URL'), // Neon fournit l'URL complète
    'host' => env('DB_HOST', '127.0.0.1'),
    'port' => env('DB_PORT', '5432'),
    'database' => env('DB_DATABASE', 'leopardo_rh'),
    'username' => env('DB_USERNAME', ''),
    'password' => env('DB_PASSWORD', ''),
    'charset' => 'utf8',
    'prefix' => '',
    'schema' => 'public',
    'sslmode' => 'require', // Obligatoire pour Neon
    'options' => extension_loaded('pdo_pgsql') ? [
        PDO::ATTR_PERSISTENT => false, // PAS de connexions persistantes (pooling)
    ] : []
]
```

Stratégie de migrations — règles absolues

Règle critique en déploiement zero-downtime : les migrations doivent être backward-compatible. L'ancienne version du code doit fonctionner pendant que la migration tourne ET pendant que la nouvelle version démarre. Toute migration breaking nécessite un processus en 3 étapes sur 3 déploiements séparés.

Opération	Backward compatible ?	Procédure
ADD COLUMN (nullable)	✓ Oui — migration safe	1 migration, 1 déploiement. L'ancienne version ignore la colonne.
ADD COLUMN (NOT NULL + default)	✓ Oui si DEFAULT fourni	1 migration, 1 déploiement. Le DEFAULT comble les anciennes lignes.
CREATE TABLE	✓ Oui	1 migration, 1 déploiement.
ADD INDEX	✓ Oui — non-blocking en Postgres	CREATE INDEX CONCURRENTLY dans la migration.
DROP COLUMN	✗ Non — breaking	Étape 1 : supprimer la référence dans le code. Étape 2 : déployer. Étape 3 : migration DROP COLUMN.
RENAME COLUMN	✗ Non — breaking	Étape 1 : add new_name + copier les données. Étape 2 : migrer le code vers new_name. Étape 3 : DROP old_name.
CHANGE TYPE	✗ Non si narrowing	Étape 1 : add colonne new type. Étape 2 : backfill + dual write. Étape 3 : switch + drop old.
CREATE UNIQUE INDEX	Attention — peut échouer si doublons	Vérifier l'absence de doublons avant. CONCURRENTLY pour éviter le lock.

CH.05

APPLICATION FLUTTER MOBILE

Build · Signing · Distribution Stores · OTA Updates

05

Architecture des builds Flutter

Cible	Commande	Artefact	Destination
Android Debug	flutter build apk --debug	app-debug.apk	Tests locaux développeur
Android Release	flutter build appbundle --release	app-release.aab	Google Play Store (obligatoire en AAB)
iOS Release	flutter build ipa --release	Runner.ipa	App Store via XCode Archive ou Fastlane
Android Staging	flutter build apk --flavor staging	app-staging.apk	Firebase App Distribution (bêta testeurs)
iOS Staging	flutter build ipa --flavor staging	Runner-staging.ipa	TestFlight (bêta testeurs)

Configuration des flavors (dev / staging / prod)

```
main_dev/staging/prod.dart — flavors
# pubspec.yaml — flavors définis dans android/app/build.gradle et ios/Runner/Info.plist

# lib/main_dev.dart
void main() {
  AppConfig.instance = AppConfig(
    flavor: Flavor.dev,
    baseUrl: 'http://localhost:8000/api/v1',
    enableLeoAI: true,
    debugBanner: true,
  );
  runApp(const LeopardoApp());
}

# lib/main_staging.dart
void main() {
  AppConfig.instance = AppConfig(
    flavor: Flavor.staging,
    baseUrl: 'https://api-staging.leopardo-rh.com/api/v1',
    enableLeoAI: true,
    debugBanner: false,
  );
  runApp(const LeopardoApp());
}

# lib/main_prod.dart
void main() {
  AppConfig.instance = AppConfig(
    flavor: Flavor.prod,
    baseUrl: 'https://api.leopardo-rh.com/api/v1',
    enableLeoAI: true,
    debugBanner: false,
  );
  runApp(const LeopardoApp());
}
```

Signing des applications — Android + iOS

Android — keystore :

```

Android signing — keystore
# Générer le keystore une fois (à conserver précieusement — JAMAIS dans le repo)
keytool -genkey -v -keystore leopardo-rh-release.jks \
  -keyalg RSA -keysize 2048 -validity 10000 \
  -alias leopardo-rh

# android/key.properties (dans .gitignore !)
storePassword=<mot de passe keystore>
keyPassword=<mot de passe clé>
keyAlias=leopardo-rh
storeFile=../leopardo-rh-release.jks

# android/app/build.gradle — référence le key.properties
def keystoreProperties = new Properties()
keystoreProperties.load(new FileInputStream(rootProject.file('key.properties')))
signingConfigs { release {
    keyAlias keystoreProperties['keyAlias']
    keyPassword keystoreProperties['keyPassword']
    storeFile file(keystoreProperties['storeFile'])
    storePassword keystoreProperties['storePassword']
} }

```

Le fichier leopardo-rh-release.jks et key.properties ne doivent JAMAIS être committés dans le repository GitHub. Ils sont stockés dans GitHub Secrets (base64 encodés) et décodés pendant le CI/CD. Si le keystore est perdu, l'app ne peut plus être mise à jour sur le Play Store — il faut republier une nouvelle app.

Distribution — Google Play Store + Apple App Store

Étape	Google Play Store	Apple App Store
Compte développeur	25 USD une fois compte Google Play Console	99 USD/an Apple Developer Program
Artefact requis	AAB (.aab) — obligatoire depuis 2021	IPA (.ipa) — via XCode Archive
Build machine	N'importe quel OS (GitHub Actions Ubuntu OK)	Obligatoirement macOS (runner GitHub macos-latest)
Signing	Keystore JKS + key.properties	Certificat + Provisioning Profile XCode Signing (automatique recommandé)
Release tracks	Internal → Closed Testing → Open Testing → Production	TestFlight (beta) → App Store Review → Production
Revue	Automatique pour la plupart des mises à jour (< 1h)	Revue humaine Apple (1 à 3 jours)
Mise à jour OTA	Non (sauf Shorebird en Phase 2)	Non (sauf Shorebird code JS non-natif)
Rollback	Oui — republier une version antérieure	Oui — republier via TestFlight

CH.06

PIPELINE CI/CD GITHUB ACTIONS

Workflows complets · Tests · Build · Deploy automatique

06

GitHub Actions est le serveur CI/CD qui orchestre tout : tests PHP, tests Flutter, build des apps, déploiement sur Render, et à terme déploiement sur Oracle VPS. Tout le code de déploiement est dans `.github/workflows/`.

Vue d'ensemble des workflows

Fichier workflow	Déclencheur	Durée estimée	Produit
tests.yml	Push sur toute branche	4-8 min	Rapport tests Pest + badge CI
mobile-distribute.yml	Push sur staging ou tag v*	15-25 min	APK staging + Firebase App Distribution
deploy-main.yml	Push sur main (après tests OK)	6-10 min	Déploiement API sur Render
flutter-prod.yml	Tag release vX.Y.Z	20-30 min	AAB Play Store + IPA App Store (futur)
codeql.yml	Weekly + push main	10-15 min	Analyse sécurité code
package-isolation.yml (nouveau)	Push sur toute branche	2-3 min	Vérification isolation packages Flutter

Workflow principal — deploy-main.yml annoté

```
deploy-main.yml — workflow complet annoté
# .github/workflows/deploy-main.yml
name: Deploy to Production
on:
  push:
    branches: [main]

jobs:
  # ■■ Étape 1 : Tests PHP
  test-php:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_DB: leopardo_test, POSTGRES_USER: test, POSTGRES_PASSWORD: test }
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.4', extensions: pdo_pgsql, pgsql }
      - run: composer install --no-dev
      - run: cp .env.testing .env && php artisan key:generate
      - run: php artisan migrate --force
      - run: ./vendor/bin/pest --coverage --min=80

  # ■■ Étape 2 : Analyse sécurité
  security-scan:
    needs: test-php
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: composer audit # Vérifie les CVE des dépendances

  # ■■ Étape 3 : Deploy sur Render
  deploy-render:
    needs: [test-php, security-scan]
    runs-on: ubuntu-latest
    steps:
      - name: Trigger Render deploy hook
        run: |
          curl -X POST ${{ secrets.RENDER_DEPLOY_HOOK_URL }}
          # Render lance le build automatiquement
          # php artisan migrate --force est dans le buildCommand
```

GitHub Secrets — liste complète à configurer

Secret	Valeur	Utilisé dans	Comment l'obtenir
RENDER_DEPLOY_HOOK_URL	https://api.render.com/deploy/...	deploy-main.yml	Render Dashboard → Deploy Hooks
RENDER_API_KEY	rnd_xxx	Scripts de vérification de status	Render Dashboard → Account Settings

Secret	Valeur	Utilisé dans	Comment l'obtenir
DATABASE_URL_STAGING	postgresql:///...@neon.tech/staging	Tests CI staging	Neon Dashboard → Connection string
FIREBASE_CREDENTIALS	{ JSON base64 }	mobile-distribute.yml (notifs)	Firebase Console → Service Account
ANDROID_KEYSTORE_BASE64	base64 du fichier .jks	flutter-prod.yml	keytool + base64 encode
ANDROID_KEY_ALIAS	leopardo-rh	flutter-prod.yml	Valeur lors de la création keystore
ANDROID_KEY_PASSWORD	xxx	flutter-prod.yml	Mot de passe clé keystore
ANDROID_STORE_PASSWORD	xxx	flutter-prod.yml	Mot de passe keystore
APPLE_SIGNING_CERT	base64 .p12	flutter-prod.yml (futur)	Keychain Access → Export certificate
APP_STORE_CONNECT_API_KEY	base64 .p8	flutter-prod.yml (futur)	App Store Connect → API Keys
OPENAI_API_KEY	sk-xxx	Tests integration (mock en CI)	OpenAI Dashboard
ORACLE_VPS_SSH_KEY	-----BEGIN RSA PRIVATE KEY-----	Migration scripts	Oracle Cloud → Instance → SSH Key

CH.07

MISES À JOUR — ARCHITECTURE COMPLÈTE

API sans coupure · App mobile · Web · Stratégie versioning

07

Stratégie globale de mise à jour par composant

Composant	Fréquence	Process	Downtime	Rollback
API Laravel	Continu (main)	GitHub Actions → Render zero-downtime	0 seconde	Révert le commit + re-push → redéploiement auto
Base de données (migrations)	Avec chaque release API	php artisan migrate --force dans le buildCommand Render	0 seconde (si migration backward-compatible)	Script SQL de revert si migration fail
App Flutter Android	Hebdomadaire ou à la demande	GitHub Actions → AAB → Play Store (revue < 1h)	N/A	Republier version précédente via Play Console
App Flutter iOS	Hebdomadaire ou à la demande	GitHub Actions → IPA → TestFlight → App Store (revue 1-3j)	N/A	Republier version précédente via App Store Connect
Dashboard Web Blade	Avec chaque release API (même codebase)	Inclus dans le déploiement API Render	0 seconde	Revert commit API
Packages Flutter (modules)	Indépendant par module	Version du package dans pubspec.yaml → rebuild app	N/A (Phase 2 Shorebird)	Downgrade la version du package
Variables d'environnement	Ponctuellement	Render Dashboard (API restart automatique) ou GitHub Secrets	~30 secondes (restart Render)	Restaurer la valeur précédente

Versioning sémantique — convention

Leopardo RH suit le versioning sémantique **MAJOR.MINOR.PATCH** pour les releases :

- **PATCH (0.1.0 → 0.1.1)** : corrections de bugs, hotfixes. Déploiement immédiat. Pas de revue app stores si c'est uniquement API.
- **MINOR (0.1.0 → 0.2.0)** : nouvelles features backward-compatible. Déploiement normal via CI/CD. Inclut souvent une mise à jour app mobile.
- **MAJOR (0.x.0 → 1.0.0)** : breaking changes API (V1 → V2). Déploiement planifié avec communication aux clients. Période de transition V1+V2 en parallèle.

Process de release — hotfix vs feature

Process gitflow simplifié – hotfix vs feature

```
# ■■ HOTFIX (bug critique en production) ■■ durée : 30-60 min
git checkout main
git checkout -b hotfix/v0.1.1-invoice-fix
# ... corriger le bug ...
git commit -m 'fix: corriger calcul TTC factures (issue #42)'
git push origin hotfix/v0.1.1-invoice-fix
# → GitHub Actions lance les tests automatiquement
# → Si tests OK → merger dans main → déploiement Render automatique
git tag v0.1.1 && git push --tags

# ■■ FEATURE RELEASE (nouvelle version) ■■ durée : 1-2 semaines
git checkout -b feature/module-cameras
# ... développement ...
git push origin feature/module-cameras
# → Pull Request vers staging → revue code → merge staging
# → Tests sur staging avec vrais utilisateurs bêta
# → Pull Request vers main → merge → déploiement prod automatique
git tag v0.2.0 && git push --tags
# → GitHub Actions flutter.prod.yml → build AAB + IPA → soumission stores
```

Mise à jour de l'app mobile sans passer par les stores (Phase 2)

Shorebird permet de pousser des mises à jour du code Dart directement sur les appareils (comme CodePush React Native) sans passer par la revue des stores. Uniquement pour le code Dart non-natif. Actif en Phase 2. Pour l'instant, toutes les mises à jour passent par Play Store / App Store.

Shorebird – OTA updates (Phase 2)

```
# Installation (Phase 2 uniquement – à ne pas faire maintenant)
curl --proto '=https' --tlsv1.2 https://raw.githubusercontent.com/shorebirdtech/install/main/install.sh -sSf | bash
shorebird init

# Publier une mise à jour OTA (Over-The-Air)
shorebird patch android --release-version=1.0.0 # Patch la release 1.0.0 en prod
shorebird patch ios --release-version=1.0.0

# Les utilisateurs reçoivent la mise à jour
# au prochain démarrage de l'app (en arrière-plan)
# Sans passer par les stores. Sans revue Apple/Google.
# Contrainte iOS : uniquement code Dart – pas de plugins natifs
```

CH.08

BACKUP & RESTAURATION

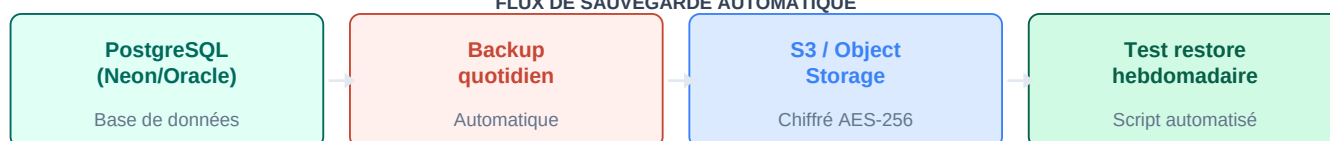
Stratégie 3-2-1 · PostgreSQL · Fichiers · Test restore

08

Stratégie de backup 3-2-1 : 3 copies des données, sur 2 supports différents, dont 1 hors-site. Pour Leopard RH : (1) Base Neon avec point-in-time recovery, (2) Dump quotidien sur S3, (3) Dump hebdomadaire sur stockage secondaire (Oracle Object Storage ou rclone vers autre provider).

Vue du flux de sauvegarde

FLUX DE SAUVEGARDE AUTOMATIQUE



Flux automatisé quotidien : PostgreSQL → pg_dump → S3 chiffré → test restore hebdomadaire.

Backup PostgreSQL — scripts complets

```
backup-db.sh — script de backup quotidien

#!/bin/bash
# scripts/backup/backup-db.sh
# À planifier via Render Cron Job ou crontab Oracle VPS

set -euo pipefail

DATE=$(date +%Y-%m-%d-%H%M)
BACKUP_FILE="leopardo-rh-{$DATE}.dump"
S3_BUCKET="s3://leopardo-rh-backups"
RETENTION_DAYS=30

# 1. Créer le dump PostgreSQL (format custom = compressé + rapide à restore)
pg_dump \
  --format=custom \           # Format binaire compressé
  --verbose \
  --no-password \
  --file="/tmp/{$BACKUP_FILE}" \
  "{$DATABASE_URL}"

# 2. Chiffrer le dump (AES-256)
openssl enc -aes-256-cbc -pbkdf2 \
  -in "/tmp/{$BACKUP_FILE}" \
  -out "/tmp/{$BACKUP_FILE}.enc" \
  -pass "pass:{$BACKUP_ENCRYPTION_KEY}"

# 3. Upload sur S3
aws s3 cp "/tmp/{$BACKUP_FILE}.enc" \
  "{$S3_BUCKET}/daily/{$BACKUP_FILE}.enc" \
  --storage-class STANDARD_IA      # Moins cher pour les archives

# 4. Supprimer les backups > RETENTION_DAYS jours
aws s3 ls "{$S3_BUCKET}/daily/" | while read -r line; do
  createDate=$(echo $line | awk '{print $1}')
  olderThan=$(date -d "{$RETENTION_DAYS} days ago" +%Y-%m-%d 2>/dev/null || \
    date -v-{$RETENTION_DAYS}d +%Y-%m-%d) # macOS compatible
  if [[ $createDate < $olderThan ]]; then
    fileName=$(echo $line | awk '{print $4}')
    aws s3 rm "{$S3_BUCKET}/daily/{$fileName}"
  fi
done

# 5. Nettoyer les fichiers temporaires
rm -f "/tmp/{$BACKUP_FILE}" "/tmp/{$BACKUP_FILE}.enc"
echo "Backup {$DATE} completed successfully"
```

Script de test de restauration — hebdomadaire

```
test-restore.sh — test de restauration hebdomadaire

#!/bin/bash
# scripts/backup/test-restore.sh
# Valide que le dernier backup peut être restauré
# À lancer chaque dimanche via cron

# 1. Télécharger le dernier backup
LATEST=$(aws s3 ls s3://leopardo-rh-backups/daily/ --recursive | sort | tail -n1 | awk '{print $4}')
aws s3 cp "s3://leopardo-rh-backups/${LATEST}" /tmp/restore-test.dump.enc

# 2. Déchiffrer
openssl enc -d -aes-256-cbc -pbkdf2 \
  -in /tmp/restore-test.dump.enc \
  -out /tmp/restore-test.dump \
  -pass "pass:${BACKUP_ENCRYPTION_KEY}"

# 3. Restaurer dans une DB de test (jamais en production !)
pg_restore \
  --no-owner --no-privileges \
  --dbname="${DATABASE_URL_TEST_RESTORE}" \
  /tmp/restore-test.dump

# 4. Vérifier l'intégrité basique
COMPANY_COUNT=$(psql ${DATABASE_URL_TEST_RESTORE} -t -c 'SELECT COUNT(*) FROM companies;')
if [ "$COMPANY_COUNT" -gt 0 ]; then
  echo "RESTORE OK — ${COMPANY_COUNT} companies restaurées"
  # Envoyer notification Slack/email de succès
else
  echo "RESTORE FAILED — 0 companies"
  # Envoyer alerte critique
  exit 1
fi
rm -f /tmp/restore-test.dump*
```

Planification des backups — crontab

```
crontab — planification automatique des backups
# crontab -e (sur le VPS Oracle ou Render Cron Job)

# Backup quotidien à 2h00 UTC (4h00 heure Algérie)
0 2 * * * /home/deploy/scripts/backup/backup-db.sh >> /var/log/backup.log 2>&1

# Test de restauration chaque dimanche à 3h00 UTC
0 3 * * 0 /home/deploy/scripts/backup/test-restore.sh >> /var/log/restore-test.log 2>&1

# Backup hebdomadaire complet (avec fichiers S3) chaque lundi à 1h00 UTC
0 1 * * 1 /home/deploy/scripts/backup/full-backup.sh >> /var/log/backup-full.log 2>&1

# Nettoyage des logs de backup > 90 jours
0 4 1 * * find /var/log -name 'backup*.log' -mtime +90 -delete
```

Matrice de récupération (RTO/RPO)

Scénario	RPO (perte max de données)	RTO (temps de rétablissement)	Procédure
Crash API (Render)	0 (stateless API)	< 2 minutes (Render restart auto)	Render redémarre le service automatiquement. Zéro intervention.
Corruption DB (partielle)	< 1 heure (backup toutes les heures en prod VPS)	15-30 minutes	Neon PITR (point-in-time) ou restore du dernier dump horaire.
Suppression accidentelle données	< 24 heures (backup quotidien)	30-60 minutes	Restore du backup quotidien sur DB de test, extraire les lignes, réinsérer.
Perte totale du VPS Oracle	< 24 heures	2-4 heures	Nouveau VPS Oracle, restaurer backup S3, relancer Docker Compose.
Perte du compte Neon	< 24 heures	4-8 heures	Backup S3 → nouveau PostgreSQL → restore → reconfigurer Render.
Perte du keystore Android	Irrecoverable (sans backup)	Publication d'une nouvelle app (semaines)	Backuper le keystore dans 2 endroits séparés dès aujourd'hui.

CH.09

MIGRATION VERS VPS ORACLE CLOUD

Pourquoi Oracle · Plan complet · Checklist · Timeline

09

La migration vers Oracle Cloud VPS est la cible de production. Oracle propose un Always Free Tier très généreux (2 vCPU ARM + 24GB RAM + 200GB stockage) sans limite de temps. C'est suffisant pour héberger l'API Laravel, PostgreSQL, MediaMTX, et Nginx sur un seul serveur — avec capacité de scale vertical immédiate si besoin.

Pourquoi Oracle Cloud — comparaison honnête

Critère	Render + Neon actuel	Oracle VPS cible	Avantage
Coût mensuel	~68 \$/mois (staging+prod)	~0-20 \$/mois (Always Free)	Oracle — 48+ \$/mois d'économie
Contrôle infra	Limité (PaaS managé)	Total (accès root SSH)	Oracle
PostgreSQL	Neon (externe, limité)	Self-hosted (illimité, tunable)	Oracle
MediaMTX (caméras)	Service séparé nécessaire	Sur le même VPS	Oracle
Scaling	Vertical limité (Render plans)	Vertical easy (shape change Oracle)	Oracle
Région	Frankfurt (Render EU)	Frankfurt (Oracle EU) — même région	Équivalent
Fiabilité (SLA)	99.9% (Render)	99.95% (Oracle SLA)	Oracle légèrement
Déploiement	Automatique (GitHub → Render hook)	Script Ansible/Docker Compose	Render plus simple
Maintenance	Zéro (Render gère)	À gérer (OS updates, security patches)	Render
Backup	Neon PITR inclus	À configurer soi-même (scripts Ch.08)	Render plus simple

Conclusion : Oracle est clairement meilleur financièrement et techniquement pour la production. Le seul coût réel est la maintenance infrastructure (2-4h/mois estimé pour un VPS bien configuré). Pour le phase dev/staging actuelle, Render + Neon restent parfaitement adaptés — pas besoin de migrer maintenant.

Architecture cible sur Oracle VPS

Structure du VPS Oracle — services déployés

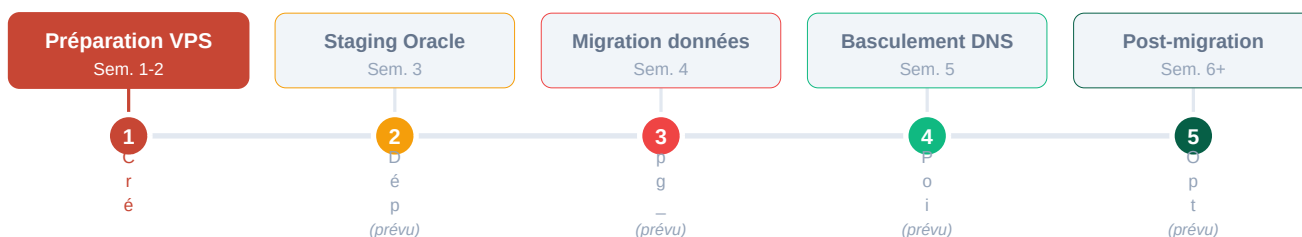
Stack déployée sur un seul VPS Oracle ARM (Ampere A1)

4 vCPU + 24GB RAM + 200GB Block Storage (Always Free)

oracle-vps/

```
■■■■ nginx/ # Reverse proxy + SSL termination (Let's Encrypt)
■ ■■■■ api.leopardo-rh.com # → localhost:8000 (Laravel)
■ ■■■■ app.leopardo-rh.com # → localhost:3000 (Dashboard Blade si séparé)
■ ■■■■ proxy.leopardo-rh.com # → localhost:8889 (MediaMTX WebRTC)
■■■■ php-fpm/ # PHP 8.4 FPM (meilleur que php artisan serve)
■ ■■■■ Laravel 11 API # /var/www/leopardo-rh/
■■■■ postgresql/ # PostgreSQL 16 self-hosted
■ ■■■■ Port 5432 (local only, pas exposé sur internet)
■ ■■■■ pgBouncer # Connection pooling (port 6432)
■■■■ mediamtx/ # Proxy caméras RTSP → WebRTC
■ ■■■■ Port 8889 (WebRTC, exposé via Nginx)
■■■■ coturn/ # TURN server pour NAT traversal caméras
■ ■■■■ Port 3478 UDP (exposé directement, pas via Nginx)
■■■■ redis/ # Cache + Queue Laravel (en prod)
■ ■■■■ Port 6379 (local only)
■■■■ supervisor/ # Gestion des processus
■■■■ php-fpm # Workers PHP
■■■■ laravel-queue-worker # Jobs async (emails, notifications)
■■■■ mediamtx # Service caméras
```

Timeline de migration — 4 phases



Phase active (violet) = préparation VPS. Les 4 phases suivantes = migration progressive avec rollback possible à chaque étape.

Checklist de migration — complète

#	Action	Responsable	Validé par	Critère de succès
1	Créer l'instance Oracle A1 Always Free dans la région Frankfurt	Ops	Dev	SSH opérationnel depuis la machine de dev
2	Configurer le firewall Oracle (ports 80, 443, 3478 UDP)	Ops	Dev	curl https://oracle-ip retourne quelque chose
3	Installer Docker + Docker Compose sur le VPS	Ops	Dev	docker --version retourne 24+

#	Action	Responsable	Validé par	Critère de succès
4	Cloner le repo sur le VPS + configurer .env production	Dev	Dev	Toutes les variables d'env présentes
5	Installer PostgreSQL 16 + pgBouncer	Ops	Dev	psql -c 'SELECT 1' retourne 1
6	Configurer Nginx + Let's Encrypt SSL pour les 3 domaines	Ops	Dev	https fonctionne, certificat valide
7	Déployer l'API Laravel sur Oracle (staging)	Dev	Dev	GET /api/health retourne 200
8	Faire tourner les tests Pest contre Oracle staging	Dev	Dev	0 test en échec
9	pg_dump de Neon → restaurer sur Oracle PostgreSQL	Ops	Dev	Compte des lignes identique avant/après
10	Configurer MediaMTX + coturn sur Oracle	Dev	Dev	Test caméra WebRTC depuis mobile 4G
11	Configurer les backups automatiques (Ch.08)	Ops	Dev	Premier backup S3 généré avec succès
12	Configurer Supervisor (php-fpm, queue, mediamtx)	Ops	Dev	ps aux grep supervisord retourne actif
13	Mettre à jour le GitHub Actions deploy (Oracle au lieu de Render)	Dev	Dev	Push test → déploiement Oracle OK
14	24h de dual-write (Render + Oracle en parallèle)	Ops	Dev	Pas de divergence de données
15	Basculer les DNS (api.leopardo-rh.com → IP Oracle)	Ops	Manager	curl https://api.leopardo-rh.com/api/health → Oracle
16	Monitoring intensif 48h	Dev	Dev	0 erreur 5xx, latence < 200ms p95
17	Désactiver Render (après 2 semaines de stabilité)	Ops	Manager	Économie de 60+\$/mois confirmée

CH.10

SÉCURITÉ & SECRETS

Variables d'env · Chiffrement · Accès · Rotation

10

Règle absolue : aucun secret (clé API, mot de passe, token) ne doit jamais apparaître dans le code source, dans les logs, ou dans un message d'erreur retourné au client. Tous les secrets passent par les variables d'environnement ou les secrets managers.

Niveaux de sécurité par couche

- **Réseau** : TLS 1.3 obligatoire sur tous les endpoints. HSTS header. Pas d'accès HTTP non-chiffré en production. PostgreSQL accessible uniquement en localhost sur le VPS (pas exposé sur internet).
- **Application** : Sanctum tokens avec expiration (config sanctum.expiration). Rate limiting sur tous les endpoints sensibles. CORS configuré pour n'accepter que les domaines Leopardo RH.
- **Base de données** : Connexion SSL obligatoire (sslmode=require). Utilisateur DB avec uniquement les permissions nécessaires (pas de SUPERUSER). Colonnes sensibles chiffrées (IBAN, rtsp_url, account_number) via `Crypt::encryptString()`.
- **Secrets** : GitHub Secrets pour le CI/CD. Variables d'environnement Render pour l'API. Jamais dans le code. Rotation trimestrielle recommandée pour les clés API externes.
- **Accès SSH (Oracle VPS)** : Authentification par clé SSH uniquement (pas de mot de passe). Clé dans GitHub Secrets pour le CI/CD. Port SSH non-standard (ex: 2222 au lieu de 22). Fail2ban pour bloquer les scans.

Procédure de rotation des secrets

Secret	Fréquence recommandée	Procédure	Impact si compromis
APP_KEY Laravel	Jamais (sauf compromission)	Nouvelle valeur → toutes les sessions invalidées → re-login obligatoire	Chiffrement compromis — invalider immédiatement
Tokens Sanctum utilisateurs	Automatique (expiration config)	<code>php artisan sanctum:prune-expired</code>	Accès non autorisé possible jusqu'à expiration
OPENAI_API_KEY / ANTHROPIC	Trimestriel ou si fuite	Nouvelle clé → mettre à jour Render env → vérifier les factures d'usage	Coûts IA non contrôlés
Mot de passe PostgreSQL	Semestriel	Nouveau password Neon → DATABASE_URL mise à jour → restart Render	Accès DB non autorisé
MEDIAMTX_INTERNAL_SECRET	Semestriel	Nouveau secret → mettre à jour MediaMTX yml et Render env	Accès flux caméras non autorisé
Clé backup S3 (BACKUP_KEY)	Annuel	Nouvelle clé → tester que les anciens backups sont toujours déchiffrables	Perte d'accès aux backups

Secret	Fréquence recommandée	Procédure	Impact si compromis
SSH Key Oracle VPS	Annuel ou si compromission	Générer nouvelle paire → ajouter sur Oracle → supprimer l'ancienne	Accès root au VPS

Configuration Nginx sécurisée — Oracle VPS

```
nginx.conf — configuration sécurisée production
# /etc/nginx/sites-available/api.leopardo-rh.com
server {
    listen 443 ssl http2;
    server_name api.leopardo-rh.com;

    # SSL — Let's Encrypt via certbot
    ssl_certificate /etc/letsencrypt/live/api.leopardo-rh.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.leopardo-rh.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3; # Pas de TLS 1.0/1.1
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256;

    # Security headers
    add_header Strict-Transport-Security 'max-age=31536000; includeSubDomains' always;
    add_header X-Frame-Options DENY always;
    add_header X-Content-Type-Options nosniff always;
    add_header Referrer-Policy 'strict-origin-when-cross-origin' always;

    # Rate limiting
    limit_req_zone $binary_remote_addr zone=api:10m rate=100r/m;
    limit_req zone=api burst=20 nodelay;

    location / {
        proxy_pass http://127.0.0.1:8000; # PHP-FPM ou artisan serve
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

# Redirection HTTP → HTTPS
server { listen 80; server_name api.leopardo-rh.com;
    return 301 https://$server_name$request_uri; }
```

Résumé — État actuel vs Cible Oracle

Composant	Maintenant (Dev)	Cible Production (Oracle VPS)	Statut
API Laravel	Render Web Service	Nginx + PHP-FPM sur Oracle VPS	PLANIFIÉ
PostgreSQL	Neon.tech (serverless)	PostgreSQL 16 self-hosted + pgBouncer	PLANIFIÉ

Composant	Maintenant (Dev)	Cible Production (Oracle VPS)	Statut
Cache	File driver	Redis sur Oracle VPS	PLANIFIÉ
Queue	Sync (dans la requête)	Laravel Queue Worker + Redis	PLANIFIÉ
MediaMTX	Service séparé	Sur le VPS Oracle	PLANIFIÉ
coturn (TURN)	Service séparé	Sur le VPS Oracle	PLANIFIÉ
CI/CD	GitHub Actions → Render hook	GitHub Actions → SSH Oracle deploy	PLANIFIÉ
SSL	Render gère	Let's Encrypt + certbot auto-renew	PLANIFIÉ
Backup DB	Neon PITR	pg_dump quotidien → S3 + PITR PostgreSQL	PLANIFIÉ
App mobile iOS	Build local	GitHub Actions (macOS runner)	EN COURS
App mobile Android	Build local	GitHub Actions (Ubuntu runner)	ACTIF
Dashboard Blade	Sur Render (avec API)	Sur Oracle VPS (avec API)	Inclus dans migration

Ce document est la référence d'infrastructure de Leopard RH. Il évolue à chaque changement d'architecture significatif. Version 1.0 — Avril 2026 — Équipe Technique Leopard RH. La migration Oracle est planifiée pour la Phase Production après la bêta.